

Pemrograman Berorientasi Objek



**Polymorphism, Static, &
Collection**

Polymorphism is achieved by Casting

- ▶ Casting is taking an Object of one particular type and "turning it into" another Object type
 - Type Casting
 - Object Casting

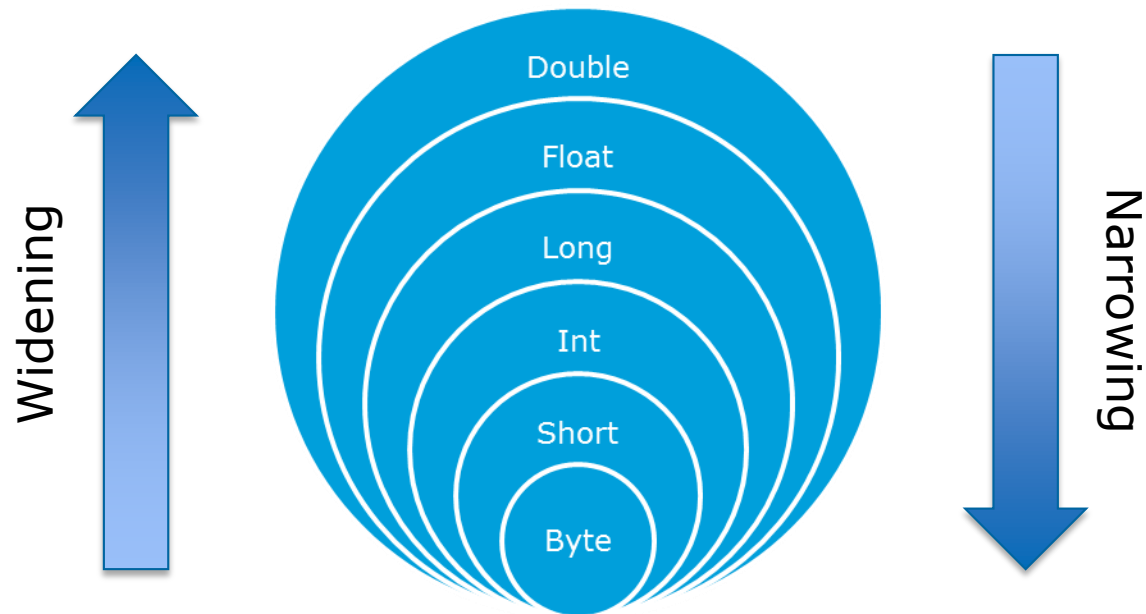


Type Casting

- ▶ Assigning a value of one type to a variable of another type
- ▶ Widening Casting
 - Implicit
 - Automatic type conversion
 - The two types must be compatible
 - The target type is larger than the source type
- ▶ Narrowing Casting
 - Explicit



Type Casting

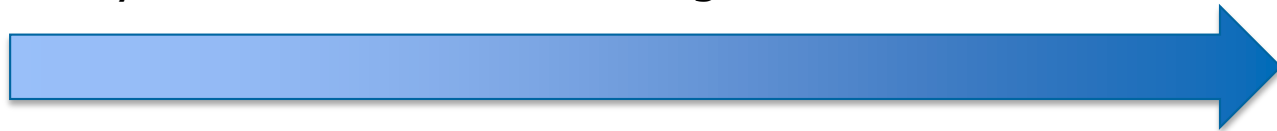


Widening Casting

```
public class Driver{  
    public static void main(String args[]){  
        int i = 50;  
        long l = i;  
        float f = l;  
  
        System.out.println("int value: " + i);  
        System.out.println("long value: " + l);  
        System.out.println("float value: " + f);  
    }  
}
```

```
> int value: 50  
> long value: 50  
> float value: 50
```

byte → short → int → long → float → double



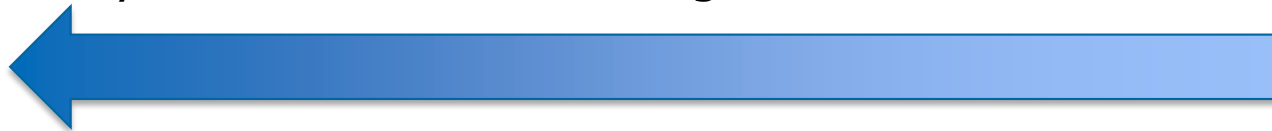
Widening

Narrowing Casting

```
public class Driver{  
    public static void main(String args[]){  
        double d = 25.16;  
        long l = (long)d;  
        int i = (int)l;  
  
        System.out.println("double value: " + d);  
        System.out.println("long value: " + l);  
        System.out.println("int value: " + i);  
    }  
}
```

```
> double value: 25.16  
> long value: 25  
> int value: 25
```

byte ← short ← int ← long ← float ← double



Narrowing

Object Casting

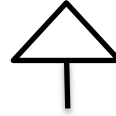
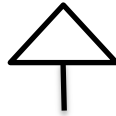
- ▶ Turn an object of a class to another class
- ▶ Both Classes should have inheritance or implement relationship
- ▶ Upcasting
 - Child object casted to parent class reference variable
- ▶ Downcasting
 - Object casted back to child class reference variable
- ▶ Virtual Method Invocation

Virtual Method Invocation

- ▶ Virtual method invocation is the form of **Upcasting**
- ▶ At the time of the object that has been created calling overridden method in the parent class, the Java compiler will do the invocation (call) to the overriding method in a subclass, which is supposed to be called is overridden

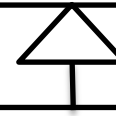
Example

```
public class Parent{  
    public String toString(){  
        return "this is class Parent";  
    }  
}
```



```
public class ChildB extends Parent{  
    public String toString(){  
        return "this is class Child B";  
    }  
}
```

```
public class ChildA extends Parent{  
    public String toString(){  
        return "this is class Child A";  
    }  
}
```



```
public class GrandChildA extends ChildA{  
    public String toString(){  
        return "this is class Grand Child";  
    }  
}
```

Example

```
public class Driver{  
    public static void main(String args[]){  
        Parent p = new Parent();  
        ChildA cA = new ChildA();  
        ChildB cB = new ChildB();  
        GrandChildA gC = new GrandChildA();  
  
        System.out.println(p.toString());  
  
        System.out.println(cA.toString());  
  
        System.out.println(cB.toString());  
  
        System.out.println(gC.toString());  
    }  
}
```

```
> this is method Parent  
  
> this is method Child A  
  
> this is method Child B  
  
> this is method Grand Child
```

Example Upcasting/VMI

```
public class Driver{  
    public static void main(String args[]){  
        Parent castP;  
  
        castP = new Parent();  
        System.out.println(castP.toString());  
  
        castP = new ChildA();  
        System.out.println(castP.toString());  
  
        castP = new ChildB();  
        System.out.println(castP.toString());  
  
        castP = new GrandChildA();  
        System.out.println(castP.toString());  
  
    }  
}
```

```
> this is method Parent  
  
> this is method Child A  
  
> this is method Child B  
  
> this is method Grand Child
```

Example Upcasting/VMI

```
public class Driver{  
    public static void main(String args[]){  
        Parent castP;  
        GrandChildA gC = new GrandChildA();  
        ChildB cB = new ChildB();  
  
        castP = gC;  
        System.out.println(castP.toString());  
  
        castP = cB;  
        System.out.println(castP.toString());  
    }  
}
```

> this is method Grand Child

> this is method Child B

Upcasting

- ▶ `castP = new ChildA();`
- ▶ Object `castP` has a behavior that is in accordance with the runtime type, not the compile type
- ▶ When compile-time `castP` is a Parent
- ▶ When runtime `castP` is ChildA
- ▶ Therefore
 - `castP` can only access variable Parent
 - `castP` can only access method ChildA

Example

```
public class Parent{  
    protected int number = 10;  
    public String toString(){  
        return "Parent " + number;  
    }  
}
```

```
public class ChildB extends Parent{  
    protected int number = 30;  
    public String toString(){  
        return "Child B " + number;  
    }  
    public String methodB(){  
        return "method Child B";  
    }  
}
```

```
public class ChildA extends Parent{  
    protected int number = 20;  
    public String toString(){  
        return "Child A " + number;  
    }  
    public String methodA(){  
        return "method Child A";  
    }  
}
```

```
public class GrandChildA extends ChildA{  
    protected int number = 40;  
    public String toString(){  
        return "Grand Child " + number;  
    }  
    public String methodGrand(){  
        return "method Grand Child A";  
    }  
}
```

Example

```
public class Driver{  
    public static void main(String args[]){  
        Parent p = new Parent();  
        ChildA cA = new ChildA();  
        ChildB cB = new ChildB();  
        GrandChildA gC = new GrandChildA();  
        Parent castP;  
  
        System.out.println(p.toString());  
        System.out.println(cA.toString());  
        System.out.println(cB.toString());  
        System.out.println(gC.toString());  
    }  
}
```

```
> Parent 10  
> Child A 20  
> Child B 30  
> Grand Child 40
```

Example

```
public class Driver{  
    public static void main(String args[]){  
        Parent p = new Parent();  
        ChildA cA = new ChildA();  
        ChildB cB = new ChildB();  
        GrandChildA gC = new GrandChildA();  
        Parent castP;  
  
        castP = cA;  
        System.out.println(castP.toString());  
        System.out.println(castP.number);  
  
        castP = gC;  
        System.out.println(castP.toString());  
        System.out.println(castP.number);  
    }  
}
```

```
> Child A 20  
> 10  
> Grand Child 40  
> 10
```


Example

```
public class Driver{  
    public static void main(String args[]){  
        Parent p = new Parent();  
        ChildA cA = new ChildA();  
        ChildB cB = new ChildB();  
        GrandChildA gC = new GrandChildA();  
        Parent castP;  
  
        System.out.println(cA.methodA());  
        castP = cA;  
        System.out.println(castP.methodA());  
    }  
}
```

**//compile error:
cannot find symbol**

> method Child A

Example - Illustration

```
Parent p;  
p = new Parent();  
  
ChildA cA;  
cA = new ChildA();
```



- Number = 10
- toString()

- toString() : "parent"



- Number = 20
- toString()
- methodA()

- toString() : "child a"
- methodA() : "method a"

Example - Illustration

```
p = cA ;  
System.out.println( p.toString() );  
System.out.println( p.number );
```



- Number = 10
- toString()



- Number = 20
- toString()
- methodA()

- toString() : "child a"
- methodA() : "method a"

DownCasting

- ▶ Returns the upcasted object back to its original class object
- ▶ Explicit casting

Example

```
public class Driver{  
    public static void main(String args[]){  
        Parent castP;  
  
        castP = new GrandChildA();  
        System.out.println(castP.toString());  
        System.out.println(castP.methodA());  
  
        GrandChildA castG = (GrandChildA) castP;  
        System.out.println(castG.toString());  
        System.out.println(castG.methodA());  
        System.out.println(castG.methodGrand());  
  
        ChildA castA = (ChildA) castP;  
        System.out.println(castA.toString());  
        System.out.println(castA.methodA());  
  
    }  
}
```

> Grand Child 40

> Compile error

> Grand Child 40

> Method Child A

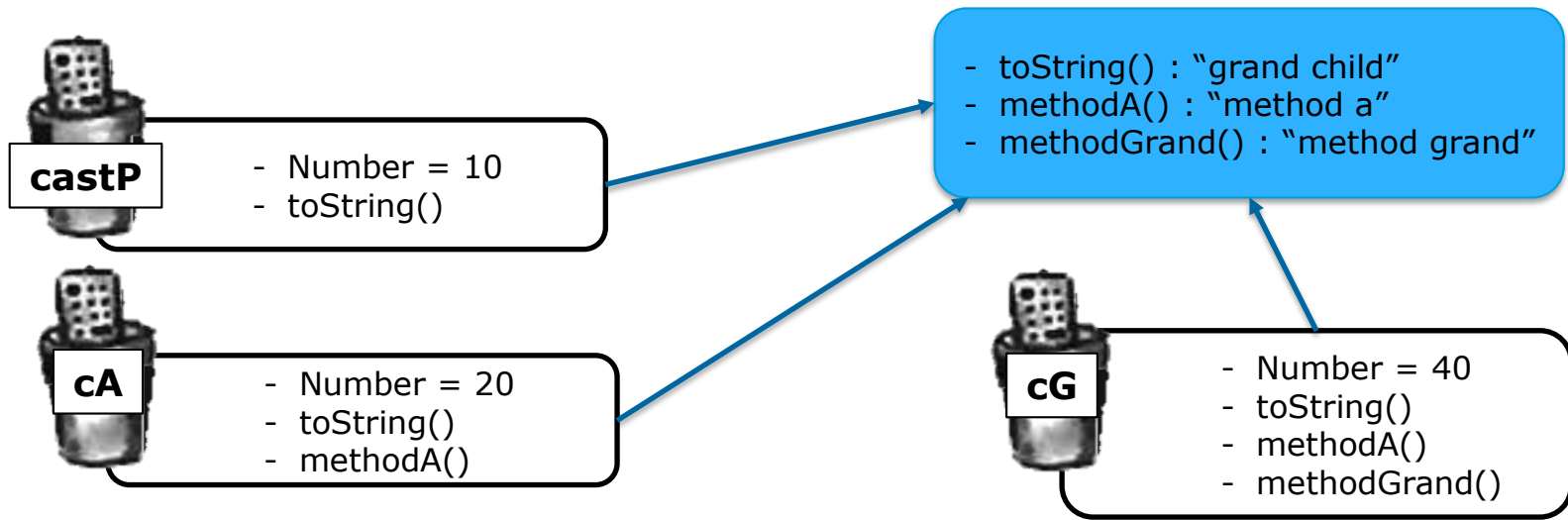
> Method Grand Child A

> Grand Child 40

> Method Child A

Example - Illustration

```
Parent castP;  
castP = new GrandChildA();  
  
ChildA cA = (ChildA) castP;  
GrandChildA cG = (GrandChildA) castP
```



Example

```
public class Driver{  
    public static void main(String args[]){  
        Parent castP;  
        castP = new GrandChildA();  
  
        System.out.println(castP.toString());  
  
        System.out.println(  
            ((ChildA)castP).methodA()  
        );  
  
        System.out.println(  
            ((GrandChildA)castP).methodGrand()  
        );  
    }  
}
```

```
> Grand Child 40  
  
> Method Child A  
  
> Method Grand Child A
```

Example

```
public class Driver{  
    public static void main(String args[]){  
        Parent castP;  
        ChildA castA;  
        GrandChildA gC;  
  
        castP = new ChildB();  
        System.out.println(castP.toString());  
  
        castA = (ChildA)castP;  
        System.out.println(castA.toString());  
        System.out.println(castA.methodA());  
    }  
}
```

> Child B 30

> **ClassCastException**

**//runtime error:
Class Cast Exception
Downcast only to
it's original object**

Example

```
public class Driver{  
    public static void main(String args[]){  
        Parent castP;  
        ChildA castA;  
        GrandChildA gC;  
  
        castP = new ChildA();  
        System.out.println(castP.toString());  
  
        gC = (GrandChildA) castP;  
        System.out.println(gC.toString());  
        System.out.println(gC.methodA());  
        System.out.println(gC.methodGrand());  
    }  
}
```

> Child A 20

> **ClassCastException**

**//runtime error:
Class Cast Exception
Downcast only to
it's original object**

Downcasting

- ▶ Can only downcast to original class or its parents' class
- ▶ To check whether the object can be casted
 - Use instanceof keyword

Keyword instanceof

- ▶ Used to test if an object is of a specified type
 - test if an object is an instance of a class,
 - an instance of a subclass,
 - or an instance of a class that implements a particular interface
- ▶ (Object instanceof Class) → boolean



Example

```
System.out.println("p instanceof Parent: "
    + (p instanceof Parent));
System.out.println("p instanceof ChildA: "
    + (p instanceof ChildA));
System.out.println("cA instanceof Parent: "
    + (cA instanceof Parent));
System.out.println("cA instanceof ChildA: "
    + (cA instanceof ChildA));

System.out.println("cB instanceof ChildA: "
    + (cB instanceof ChildA));
System.out.println("cB instanceof Parent : "
    + (cB instanceof Parent));

System.out.println("cG instanceof ChildA: "
    + (cG instanceof ChildA));
System.out.println("cG instanceof Parent: "
    + (cG instanceof Parent));
```

```
> p instanceof Parent: true
> p instanceof ChildA: false
> cA instanceof Parent: true
> cA instanceof ChildA: true

> //compile error, cannot be converted
> cB instanceof Parent : true

> cG instanceof ChildA: true
> cG instanceof Parent : true
```

Example

```
public class Driver{  
    public static void main(String args[]){  
        Parent castP;  
        ChildA castA;  
        GrandChildA gC;  
  
        castP = new ChildA();  
        System.out.println(castP.toString());  
  
        if(castP instanceof GrandChildA) {  
            gC = (GrandChildA) castP;  
            System.out.println(gC.toString());  
            System.out.println(gC.methodA());  
            System.out.println(gC.methodGrand());  
        }  
    }  
}
```

> Child A 40

>

Benefits and Downsides

- ▶ Flexibility
 - Model Object
- ▶ Heterogeneous Collection
- ▶ Polymorphic Arguments
- ▶ Run-time exception
 - `ClassCastException`

Heterogeneous Collection

- ▶ Collections of objects with different class types

```
public class Driver{  
    public static void main(String args[]){  
        Parent listP[] = new Parent[4];  
        listP[0] = new ChildB();  
        listP[1] = new ChildA();  
        listP[2] = new Parent();  
        listP[3] = new GrandChildA();  
    }  
}
```

Example

```
for(int i = 0; i < 4; i++){  
  
    System.out.println(i+" "+listP[i].toString());  
  
    if(listP[i] instanceof ChildA) {  
        ChildA cA = (ChildA)listP[i];  
        System.out.println(cA.methodA());  
    }  
  
    if(listP[i] instanceof ChildB) {  
        ChildB cB = (ChildB)listP[i];  
        System.out.println(cB.methodB());  
    }  
  
    if(listP[i] instanceof GrandChildA) {  
        GrandChildA gC = (GrandChildA)listP[i];  
        System.out.println(gC.methodGrand());  
    }  
}
```

```
> 0 Child B 30  
> method Child B  
  
> 1 Child A 20  
> method Child A  
  
> 2 Parent 10  
  
> 3 Grand Child 40  
> method Child A  
> method Grand Child A
```


Polymorphic Arguments

- ▶ Method with parent reference as parameter input

```
public void testMethod(Parent p){  
    System.out.println(p.toString());  
  
    if(p instanceof ChildA) {  
        System.out.println((ChildA)p.methodA());  
    } else if(p instanceof ChildB) {  
        System.out.println((ChildB)p.methodB());  
    } else if(p instanceof GrandChildA) {  
        System.out.println((GrandChildA)p.methodGrand());  
    }  
}
```

Static Modifier

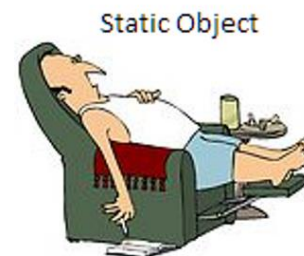
Static Object



Dynamic Object

Static Modifier

- Static variables
- Static method



Static Variables

- ▶ Create variables that will exist independently of any instances created for the class.
- ▶ Only one copy of the static variable exists regardless of the number of instances of the class.
- ▶ Also known as class variables.
- ▶ Local variables cannot be declared static

Static Variables

- ▶ One variable shared for all instances
- ▶ One value per class, instead of one value per instance

Normal Variables

```
public class Duck{  
    private int size;  
  
    public Duck(int size) {  
        this.size = size;  
    }  
  
    public int getSize(){  
        return size;  
    }  
}
```

```
public class Driver{  
    public static void main(String args[]){  
        Duck d;  
        d = new Duck(4);  
        d = new Duck(10);  
        d = new Duck(16);  
        d = new Duck(5);  
    }  
}
```

Question: how many
Ducks that have been
instantiated?

Normal Variables

```
public class Duck{
    private int size;
    private int duckCount = 0;

    public Duck(int size) {
        this.size = size;
        duckCount++;
    }

    public int getSize(){
        return size;
    }

    public int getCount(){
        return duckCount;
    }
}
```

```
public class Driver{
    public static void main(String args[]){
        Duck d;
        d = new Duck(4);
        d = new Duck(10);
        d = new Duck(16);
        d = new Duck(5);

        System.out.println(d.getCount());
    }
}
```

Output :
> 1

Static Variables

```
public class Duck{
    private int size;
    private static int duckCount = 0;

    public Duck(int size) {
        this.size = size;
        duckCount++;
    }

    public int getSize(){
        return size;
    }

    public int getCount(){
        return duckCount;
    }
}
```

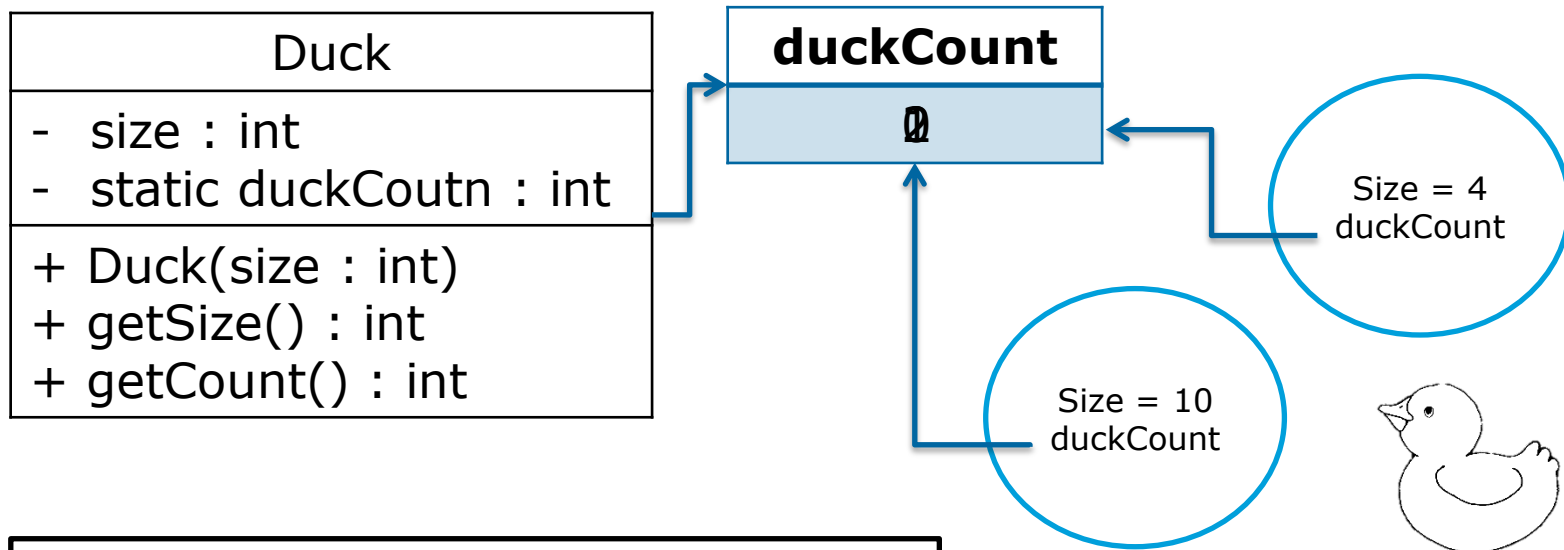
```
public class Driver{
    public static void main(String args[]){
        Duck d;
        d = new Duck(4);
        d = new Duck(10);
        d = new Duck(16);
        d = new Duck(5);

        System.out.println(d.getCount());
    }
}
```

Output :

> 4

Static Variables



- ▶ A duck object doesn't keep its own copy of duckCount
- ▶ Duck objects all share a single copy of duckCount
- ▶ Variable that lives in a CLASS, instead of in an object

Each Duck object has its own size variable, but there's only one copy of duckCount variable

Static Variables

```
public class Share {  
    private int privateInt;  
    private static int staticInt;  
  
    public Share(int pr, int st){  
        privateInt = pr;  
        staticInt = st;  
    }  
  
    public String toString(){  
        return privateInt + " " + staticInt;  
    }  
}
```

Static Variables

```
public class Driver{  
    public static void main(String args[]){  
        Share s1 = new Share(4,4);  
        System.out.println(s1.toString());  
  
        Share s2 = new Share(8,2);  
        System.out.println(s1.toString());  
        System.out.println(s2.toString());  
  
        Share s3 = new Share(6,22);  
        System.out.println(s1.toString());  
        System.out.println(s2.toString());  
        System.out.println(s3.toString());  
    }  
}
```

Output :

> 4 4

> 4 2

> 8 2

> 4 22

> 8 22

> 6 22

Static Methods

- ▶ Create methods that will exist independently of any instances created for the class
- ▶ Static methods do not use any instance variables of any object of the class they are defined in.
- ▶ Only can access static variables

Static Methods

- ▶ Static methods take all the data from parameters and compute something from those parameters, with no reference to variables
- ▶ Class variables and methods can be accessed using the class name followed by a dot and the name of the variable or method
 - Without instantiating the object

Static Methods

```
public class CounterMachine{  
    private static int counter = 0;  
  
    public static void count(){  
        counter++;  
    }  
  
    public static int getCounter(){  
        return counter;  
    }  
}
```

```
public class Driver{  
    public static void main(String args[]){  
  
        for(int i = 0; i < 10; i++){  
            if( i % 2 == 0){  
                CounterMachine.count();  
            }  
        }  
  
        System.out.println(  
            CounterMachine.getCounter());  
    }  
}
```

Output :

> 5

Static Variables and Methods

```
public class CounterMachine{  
    public static int counter = 0;  
  
    public static void count(){  
        counter++;  
    }  
}
```

```
public class Driver{  
    public static void main(String args[]){  
  
        for(int i = 0; i < 10; i++){  
            if( i % 2 == 0){  
                CounterMachine.count();  
            }  
        }  
  
        System.out.println(  
            CounterMachine.counter);  
    }  
}
```

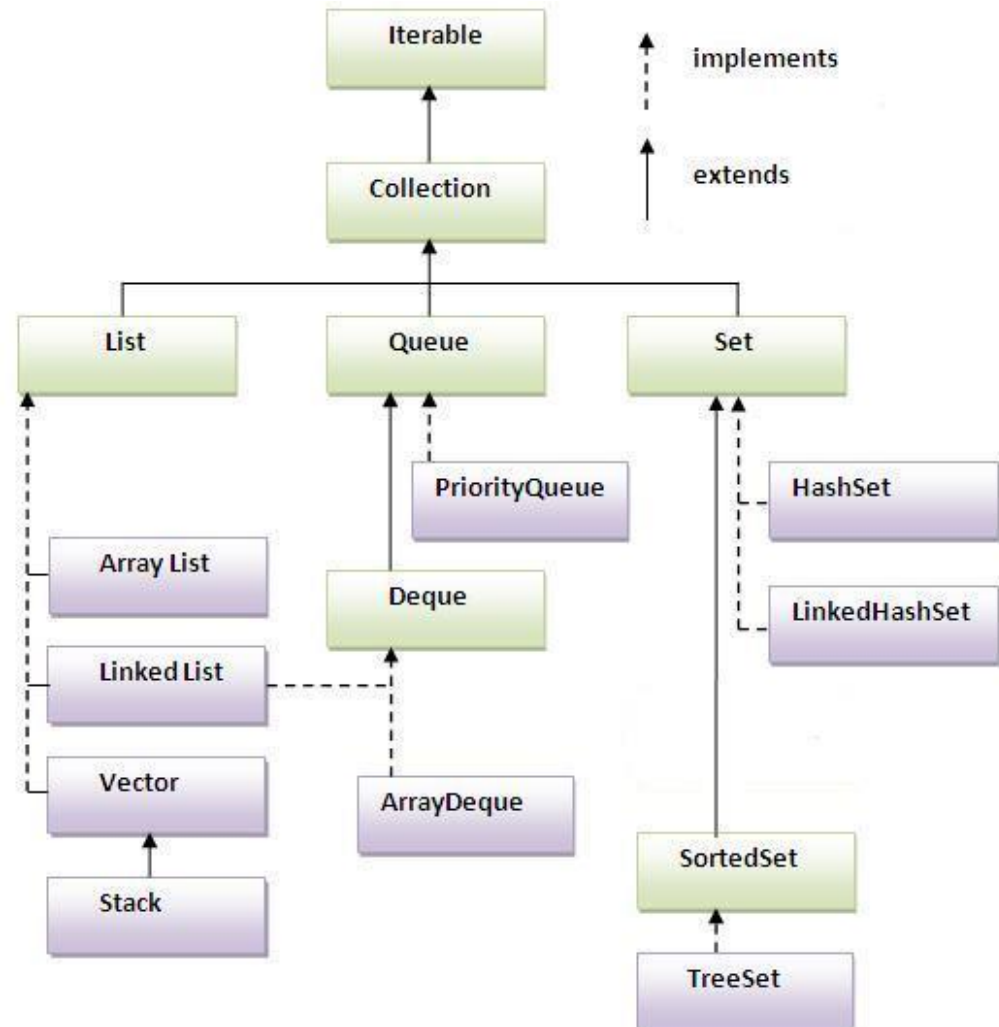
Java Collection



Java Collection

containers of Objects which by polymorphism can hold any class that derives from Object (which is actually, any class)

Using **Generics** the Collection classes can be aware of the types they store



Java Collection

- ▶ `add( item )`
- ▶ `remove( item )`
- ▶ `addAll( collection )`
- ▶ `removeAll( collection )`
- ▶ `retainAll( collection )`
- ▶ `contains( item )`

Java Collection

```
List list = new ArrayList();
```

```
// normal loop
```

```
for (int i = 0; i < list.size() ; i++ ) {  
    Object o = list.get(i);  
    System.out.println(o);  
}
```

```
// loop using for-element
```

```
for (Object o : list) {  
    System.out.println(o);  
}
```

Java Collection

// Loop using iterator

```
Iterator itr = list.iterator();  
while (itr.hasNext()) {  
    Object o = itr.next();  
    System.out.println(o);  
}
```

// loop using lambda Expression

```
list.forEach( o -> System.out.println(o));
```

// loop using reference

```
list.forEach( System.out :: println);
```

Generic Collection

- `Collection<String> str = new HashSet<String>();`
- `List<Integer> arr_i = new List ();`
- `ArrayList<Employee> emp = new ArrayList ();`

Sorting Collection – Comparable

- ▶ `Collections.sort( list )` or `list.sort( null )`
 - If the List consists of String elements, it will be sorted into alphabetical order.
 - If it consists of Date elements, it will be sorted into chronological order
 - To create a custom ordering for list of objects, the class must implement interface Comparable
 - implement method `public int compareTo( Object )`
- ▶ For arrays, use `Arrays.sort( array[] )`

Sorting Collection – Comparable

- ▶ `public int compareTo( Object o )`
 - Return < 0 if “this” object will be sorted before Object o
 - Return $= 0$ if “this” object and Object o is equal
 - Return > 0 if “this” object will be sorted after Object o
- ▶ If the parameter is either String or Date, use `compareTo`
 - return `this.getString().compareTo(o.getString());`
- ▶ If the parameter is numerical, use subtraction
 - return `this.getNumber() - o.getNumber();`

Example - sort(list of string)

```
List<String> list = new ArrayList();  
list.add("Pineapple");  
list.add("Apple");  
list.add("Orange");  
list.add("Banana");  
Collections.sort(list);  
  
int i = 0;  
for (String s : list) {  
    System.out.println("fruits " + ++i + " : " + s);  
}
```

```
String[] fruits  
    = new String[]{"Pineapple", "Apple", "Orange", "Banana"};  
Arrays.sort(fruits);  
int i = 0;  
for (String temp : fruits) {  
    System.out.println("fruits " + ++i + " : " + temp);  
}
```


Example – sort(list of object)

```
public class Employee implements Comparable<Employee> {
```

```
    private String name;  
    private double salary;
```

```
    public Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }
```

```
    public String getName() {  
        return name;  
    }
```

```
    public double getSalary() {  
        return salary;  
    }
```

```
    @Override
```

```
    public String toString() {  
        return "name=" + name + ", salary=" + salary;  
    }
```

```
    @Override
```

```
    public int compareTo(Employee t) {  
        return (this.name).compareTo(t.name);  
    }
```

```
}
```

Example – sort(list of object)

```
public static void main(String[] args) {  
    List<Employee> listEmp = new ArrayList();  
  
    listEmp.add(new Employee("bobby", 5));  
    listEmp.add(new Employee("erick", 56));  
    listEmp.add(new Employee("anna", 15));  
    listEmp.add(new Employee("rey", 25));  
  
    Collections.sort(listEmp);  
    listEmp.forEach( System.out :: println);  
}
```

name=anna, salary=15.0
name=bobby, salary=5.0
name=erick, salary=56.0
name=rey, salary=25.0

Sorting Collection - Comparator

- ▶ Comparable Interface only allows to sort a single property.
- ▶ To sort with multiple properties, you need **Comparator class**
- ▶ `Collections.sort( list, comparator )`
or `list.sort( comparator )`
- ▶ For arrays, use `Arrays.sort( array[], comparator )`

Sorting Collection - Comparator

- ▶ Create a comparator class implements Comparator interface to compare the element
 - public interface Comparator<T> {
 int compare(T object1, T object2);
}

```
public class SalaryComparator implements Comparator<Employee>{  
  
    @Override  
    public int compare(Employee t, Employee t1) {  
        return (int) (t.getSalary() - t1.getSalary());  
    }  
  
}
```

Sorting Collection Example

```
public static void main(String[] args) {  
    List<Employee> listEmp = new ArrayList();  
  
    listEmp.add(new Employee("bobby", 5));  
    listEmp.add(new Employee("erick", 56));  
    listEmp.add(new Employee("anna", 15));  
    listEmp.add(new Employee("rey", 25));  
  
    Collections.sort(listEmp);  
    for (Employee emp : listEmp) {  
        System.out.println(emp);  
    }  
  
    Collections.sort(listEmp, new SalaryComparator());  
    for (Employee emp : listEmp) {  
        System.out.println(emp);  
    }  
}
```

name=anna, salary=15.0
name=bobby, salary=5.0
name=erick, salary=56.0
name=rey, salary=25.0

name=bobby, salary=5.0
name=anna, salary=15.0
name=rey, salary=25.0
name=erick, salary=56.0

Filter elements from Collections

- ▶ Select a specific element based on parameter
 - Select all Employee with salary above 20
- ▶ Old way

```
List<Employee> temp = new ArrayList();  
for (Employee e : listEmp) {  
    if (e.getSalary() > 20) {  
        temp.add(e);  
    }  
}
```

Filter elements from Collections

- ▶ Select all Employee with salary above 20
- ▶ Using Lambda expression
 - `list.stream().filter( o -> condition )`

```
List<Employee> temp = listEmp.stream()  
    .filter(e -> e.getSalary() > 20).collect(Collectors.toList());
```

Filter elements from Collections

- ▶ Retrieve an element based on some attributes
 - Select Employee by name
- ▶ Old way

```
public Employee getEmployee(String name) {  
    for (Employee e : listEmp) {  
        if (e.getName().equals(name)) {  
            return e;  
        }  
    }  
    return null;  
}
```


Filter elements from Collections

- ▶ Select Employee by name
- ▶ Using Lambda expression
 - `list.stream().filter( o -> condition ).findFirst()`
 - `.orElse( x )`

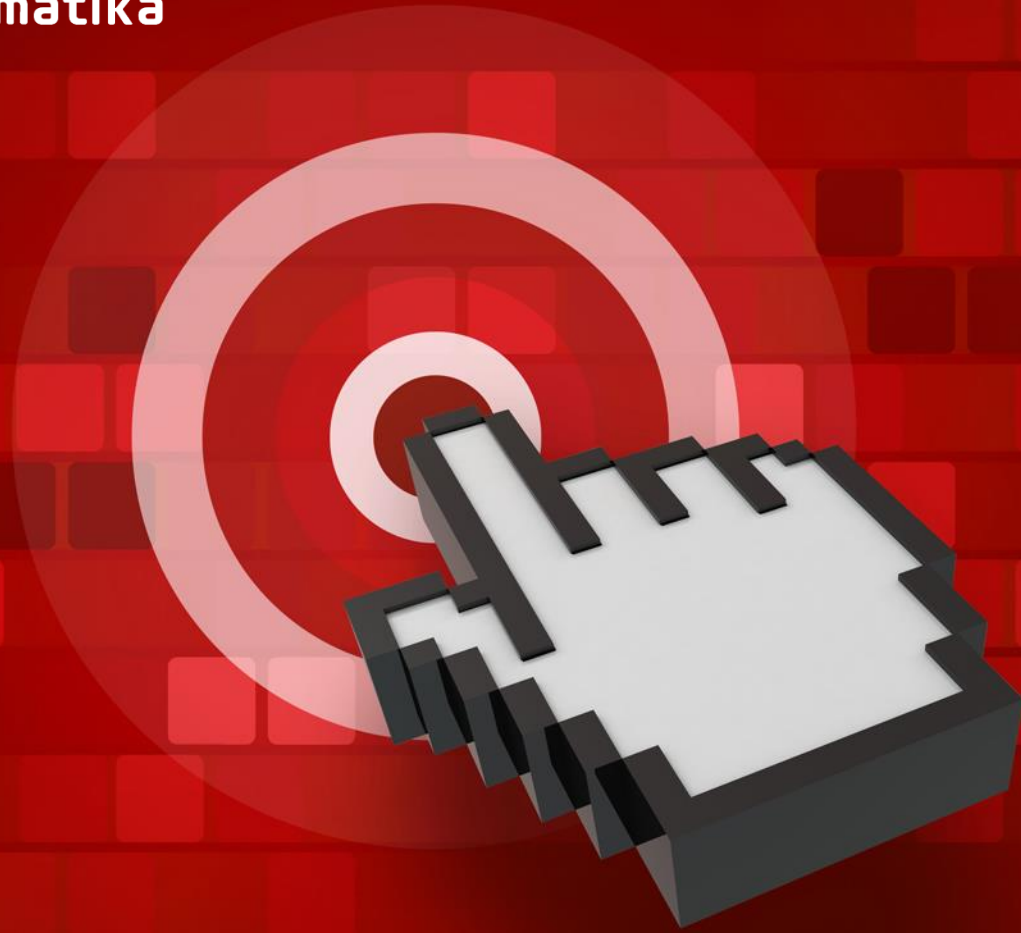
```
public Employee getEmployee(String name) {  
    return listEmp.stream()  
        .filter(o -> o.getName().equals(name))  
        .findFirst().orElse(null);  
}
```

Question?





Fakultas Informatika
School of Computing
Telkom University



THANK YOU